Smart Text Classifier: Project Report

Varolline Internship Program, AI Engineer Technical Assignment

1. Goal

Build a small, working application that classifies short text into **Complaint, Inquiry, Feedback or Other**, with a documented dataset, honest evaluation, a simple interface and a clear list of limitations.

2. Approach

Data. No suitable public dataset matched these four labels, so I created one (588 rows, 196 hand-written seed sentences, roughly balanced: 153 / 141 / 144 / 150). Seeds cover five service domains (food delivery, banking app, online shop, university portal, telecom) plus domain-free sentences, and follow written label definitions and tie-breaker rules (`data/LABELS.md`). Each seed gets up to two class-neutral variations (greeting or sign-off, casing, dropped full stop, one typo). Hard cases were added on purpose: inquiries without a question mark, polite complaints, complaints phrased as questions, feedback containing negative words, and off-topic questions labelled Other. An AI assistant helped draft the seeds; I reviewed and edited them.

Preprocessing. Lower-casing, URL and number placeholders, and turning `?` and `!` into tokens. I deliberately kept stopwords, because words such as *not*, *how* and *why* carry the label.

Model. TF-IDF with word unigrams and bigrams, followed by a classifier. I compared Multinomial Naive Bayes with Logistic Regression and chose by cross-validated macro-F1, using a simple, fast, explainable approach that suits a small dataset.

Evaluation. An 80/20 stratified split (468 train, 120 test) plus 5-fold cross-validation on the training part. Metrics: accuracy, macro-F1 and weighted-F1, with a per-class report and confusion matrix. A separate 32-sentence **challenge set** with new wording was never used for training or selection.

3. Key decisions

1. **Group-aware splitting.** Variants of the same seed are near-duplicates. A random split would put one variant in train and another in test and inflate the score. Every row carries a `group` id and the split keeps each group on one side.
2. **Challenge set.** Because the main data is partly synthetic, I added unseen, differently worded sentences to get a more honest estimate.
3. **Keep stopwords and punctuation signals.** Standard stopword removal would delete words that distinguish the classes.
4. **Refit on all data for the saved model.** Metrics come from the train/test protocol; the deployed model is then refit on the full dataset.

4. Results

Evaluation	Accuracy	Macro-F1
Grouped CV, Naive Bayes	0.664	0.659
Grouped CV, Logistic Regression	0.765	0.762
Test set, Naive Bayes	0.792	0.787
Test set, Logistic Regression (selected)	**0.867**	**0.865**
Challenge set (32 unseen), Logistic Regression	0.813	0.809

Logistic Regression outperformed Naive Bayes in both cross-validation and on the test set. On the test set, Inquiry was easiest (F1 0.94); Feedback had the lowest recall (0.70), mostly confused with Complaint because both can contain negative words. The scores are far from perfect, which is expected given the deliberately hard cases.

Error analysis (challenge set). The saved final model gets 27 of 32 correct (the 0.813 above is the train-split model, 26 of 32). Its five errors were: a polite “Is it possible to switch to the yearly plan?” labelled Feedback; the positive “Support agent was kind and explained everything patiently” labelled Complaint; the off-topic “What is the tallest mountain in the

world?” labelled Inquiry; and two noise texts (“Haha that was funny”, “Testing testing one two three”) assigned to Feedback and Complaint. The top Complaint features include function words (“and”, “was”, “my”), which shows the model partly learned that complaints are longer, a shortcut that comes from the dataset.

5. Challenges

- Making the dataset realistic rather than trivially separable (avoiding a “?” shortcut: 70% of Inquiries contain one, 22% of Other texts also do).
- Avoiding leakage between near-duplicate rows, which required group-aware splitting.
- Ambiguous cases: messages that combine a complaint and a question, resolved with written tie-breaker rules.

6. Limitations and future improvements

Limitations: small, partly synthetic dataset; style shortcuts such as text length; weak handling of off-topic questions, mixed intent and sarcasm; English only; probabilities not calibrated.

Improvements: (1) fine-tune a small transformer or use sentence embeddings for better semantic understanding; (2) collect real labelled messages, including Roman Urdu, and add a confidence threshold that routes uncertain texts to a human or to “Other”; (3) support multi-label output; (4) add an explicit out-of-scope class and calibrate probabilities.

7. Deliverables

Source code and README in the repository; model `models/classifier.joblib`; metrics and plots in `reports/`; Streamlit app (`streamlit run src/app.py`); 32 demo predictions in `reports/demo_predictions.csv`.